Hashing und Hashmaps

Dozent: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de , Raum 149B

Version: 2.2.0.1

Quelle: Die Folien sind teilweise inspiriert von oder basierend auf:

- Robert Sedgewick und Kevin Wayne, "Algorithms", Addison-Wesley, 4th Edition, 2011
- Lehrmaterial von Prof. Dr. Ritterbusch

Folien: https://delors.github.io/theo-algo-hashing/folien.de.md.html
https://delors.github.io/theo-algo-hashing/folien.de.md.html.pdf

Kontrollfragen: https://delors.github.io/theo-algo-hashing/kontrollfragen.de.md.html

Fehler melden: https://github.com/Delors/delors.github.io/issues

KI Verwendung: Bei der Erstellung der Unterlagen wurden KI Assistenten (insbesondere Claude aber ggf. auch ChatGPT, Ollama mit Gemma/LLama/Qwen oder OpenCode mit Kimi/Qwen/Deepseek...) unterstützend eingesetzt. Dies erfolgte insbesondere zur Unterstützung bei der Generierung von Grafiken (d. h. SVG Dateien), oder um sich Übersichtstabellen generieren zu lassen. Weiterhin wurde KI zur allgemeinen Qualitätssicherung eingesetzt. Inhalte, die ggf. von der KI vorgeschlagen wurden, wurden im Falle der Übernahme explizit validiert und angepasst.

1. Einführung

Komplexität der Suche in einer (*duplikatfreien*) Symboltabelle mit n Elementen

Implementation	Garantie			Durchschnittlicher Fall			Operationen auf den Schlüsseln
	Suchen	Einfügen	Löschen	Suchen	Einfügen[1]	Löschen	
sequentielle Suche einfach verkettete, unsortierte Liste	n	n	n	½ n	n	½ n	equals()
binäre Suche geordnetes, dynamisches Array	lg n	n	n	lg n	½ n	½ n	compareTo()
binärer Suchbaum	n	n	n	1,39 lg n	1,39 lg n	√ n	compareTo()

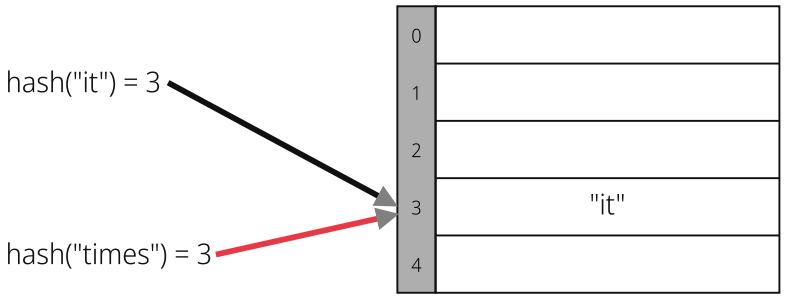
? Frage

Können wir effizienter suchen?

---

[1] Eingefügt wird nur, wenn das Element noch nicht enthalten ist!

Hashing - Grundidee



- Die Elemente werden **über den Schlüssel indexiert** in einer Tabelle **gespeichert**.

  Der Index ist eine Funktion des Schlüssels.

  In dem Beispiel sind die Schlüssel die Worte: `it` und `times`.

- Hash-Funktion[2]: Methode zur Berechnung des Array-Index aus dem Schlüssel.

Herausforderungen

1. Berechnung der Hash-Funktion.
2. Gleichheitstest: Methode zur Überprüfung, ob zwei Schlüssel gleich sind.
3. Kollisionsauflösung: Algorithmus und Datenstruktur zur Behandlung von zwei Schlüsseln, die auf denselben Array-Index hindeuten.

※ Hinweis

Klassischer Kompromiss zwischen Raum und Zeit!

Keine Platzbeschränkung:	triviale Hash-Funktion mit Schlüssel als Index.
Keine Zeitbeschränkung:	triviale Kollisionsauflösung mit sequentieller Suche.
Raum- und Zeitbeschränkung:	Hashing (die reale Welt).

---

[2] Nur im Zusammenhang mit Datenstrukturen! Kryptographische Hash-Funktionen haben bzw. erfüllen diesen Zweck nicht.

Berechnung der Hash-Funktion

Idealistisches Ziel: Die *Schlüssel gleichmäßig verwürfeln*, um einen Tabellenindex zu erzeugen.

- Effizient berechenbar.
- Jeder Tabellenindex ist für jeden Schlüssel gleich wahrscheinlich.

Im Allgemeinen gilt: Die Frage, wie man gute Schlüssel berechnet, ist ein gründlich erforschtes Problem, dass in der Praxis immer noch problematisch ist.

Beispiel: Beispiel 1. Telefonnummern.

Schlecht: die ersten drei bis fünf Ziffern

Besser: die letzten vier Ziffern

Beispiel: Beispiel 2. Sozialversicherungsnummer

Die ersten beiden Stellen bei der Sozialversicherungsnummer identifizieren den Rentenversicherungsträger.

Schlecht: die ersten beiden Ziffern

Besser: die letzten Ziffern

Achtung!

Praktische Herausforderung: für jeden Schlüsseltyp ist ein anderer Ansatz erforderlich.

2. Hashfunktionen

Grundlagen

Definition: Hashfunktion

Eine Hashfunktion $h : M \to \mathbb{Z}_n$ bildet eine Menge M mit $|M| \geq |\mathbb{Z}_n|$ auf die Zahlen $0, ..., n-1$ ab.

Eine Hashfunktion ist *surjektiv* wenn für jedes $y \in Z_n$ es ein $x \in M$ mit $h(x) = y$ gibt.

Eine Hashfunktion ist *gleichverteilt*, wenn zwei Bilder $y1, y2 \in \mathbb{Z}_n$ immer ungefähr gleich viele Urbilder haben $|h^{-1}(y1)| \approx |h^{-1}(y2)|$.

In manchen Fällen ist der Nachweis der Surjektivität nicht möglich, aber es wird vermutet. In der Praxis können oft auch Hashfunktionen verwendet werden, die nicht-surjektiv sind. Dies hat jedoch signifikante Auswirkungen auf die Effizienz der Hashtabelle.

Die Gleichverteilung muss – für das korrekte Funktionieren von Datenstrukturen, die Hashfunktionen verwenden – häufig nicht eingehalten werden. Jedoch gilt auch hier, dass dies ggf. signifikante Auswirkungen auf die Effizienz der Hashtabelle hat. Darüber hinaus ist die Beschaffenheit der konkreten Daten, die durch die Hashfunktion in einem konkreten Szenario verarbeitet werden, ggf. ebenfalls ein wichtiger Faktor bei der Wahl der Hashfunktion.

Wichtige Anforderungen an Hashes für unterschiedliche Anwendungen

Datenstrukturen:
Die Hashfunktionen müssen sehr effizient sein.

Integritätsprüfung und Signaturen:
Für die verwendeten Hashfunktionen muss gelten, dass es schwer ist:

- ein Urbild zu finden (d. h. von Y auf X zu schließen)
- zwei kollidierende Werte zu finden.

Eine effiziente Berechnung ist wünschenswert.

Die Hashfunktion MD5 ist seit 2008 und SHA1 seit 2017 für praktische Anwendungen „geknackt“.

Passwortspeicherung:
Hashfunktionen für Passwortspeicherung sollten im Wesentlichen dieselben Anforderungen erfüllen wie Hashfunktionen für Integritätszwecke, *dürfen aber nicht effizient berechenbar sein*, um Brute-Force Angriffe zu verhindern.

!! Wichtig

Im Folgenden konzentrieren wir uns auf Hashes für Datenstrukturen.

Wenn das Ziel ist, Hash-Werte mit einer bestimmten Länge (zum Beispiel 32Bit) zu berechnen, dann wären folgende Hashfunktionen denkbar:

Exemplarische Hashfunktionen

Ganze Zahlen

```
1 hash(x: u32) : u32 { // u32 == 32-Bit unsigned integer
2     return x;
3 }
```

Gleitkommazahlen

```
fn hash(x: f64) : u32 { // f64 == 64-Bit (IEEE) floating point number
    bits : u64 = f64ToBits(x); // u64 = 64-Bit (signed) integer
    // >>> ist der *unsigned right shift* Operator.
    return (u32) (bits ^ (bits >>> 32));
}
```

Zeichenketten

Horners Methode für Zeichenketten der Länge L:

$h = s[0] \cdot 31^{L-1} + \ldots + s[L-2] \cdot 31^{1} + s[L-1] \cdot 31^{0}$

```
fn hash(s: [char,4]) : u32 {
    hash: u32 = 0
    for i in 0..4 { hash = s[i] + hash * 31; }
    return hash;
}
// hash(['c','a','l','l']) = // ≙ hash("call")
//   99 · 31·31·31 + 97 · 31·31 + 108 · 31 + 108 =
//   108 + 31 · ( 108 + 31 · (97 + 31 · (99)))
```

◎ Bemerkung

Zeichen	Unicode Wert
'a'	97
'b'	98
'c'	99
⋮	⋮
'l'	108

Gängige Ansätze für Hashfunktionen

Modulo-Hashfunktion:

Sei n möglichst eine Primzahl:

$h_n^{mod}(x) = x \ mod \ n$

Bewertung

- einfach zu berechnen/sehr effizient
- surjektiv
- gleichverteilt
- wenn n keine Primzahl ist, dann kann es (leicht) passieren, dass bestimmte (Teil-)daten weniger oder keinen Einfluss auf den Hashwert haben:
  - $x \cdot 10^3 \mod 40 = 0$
  - $x \cdot 10^3 \mod 42 \in \{0, 2, 4, ..., 40\}$ Anm.: $ggT(42, 1000) = 2$
  - $x \cdot 10^3 \mod 41 \in \{0, 1, 2, 3, ..., 40\}$ Anm.: $ggT(41, 1000) = 1$

Multiplikations-Hashfunktion:

Sei c fest, oft $c = \frac{\sqrt{5}-1}{2} \approx 0,6180339887498949$:

$h_n^{mul}(x) = \lfloor n \cdot (c \cdot x - \lfloor c \cdot x \rfloor) \rfloor$

Bewertung

- nicht beweisbar surjektiv
- nur asymptotisch gleichverteilt
- Das verwendete c sollte eine gute Durchmischung der Schlüssel-Bits fördern.

  Andere irrationale Zahlen sind ggf. auch sinnvoll/möglich.
- Berechnung benötigt eine effiziente Fließkomma-Verarbeitung

✎ Übung

2.1. Zufall hashen

Stellen Sie sich vor, Sie haben 512 Bits, die unabhängig und gleichverteilt („zufällig“) sind. Sie möchten mehrere Werte davon in einer Tabelle mit 256 Einträgen speichern. Wie könnte die Hashfunktion aussehen?

2.2. URLs hashen

Eine Logging-Anwendung speichert URLs in einer Hashtabelle. Praktisch alle URLs beginnen mit `https://www.dhbw-mannheim.de/`. Tabellengröße: 4096.

Diskutieren Sie den folgenden Vorschlag: Nimm die ersten 8 Zeichen der URL und interpretiere sie als 64-Bit-Wert, dann modulo 4096.

✎ Übung

2.3. Periodische Daten verarbeiten

Sie speichern Sensordaten, die einen Zeitstempel in Sekunden seit Mitternacht enthalten (Werte 0 bis 86399). Die Geräte senden alle 60 Sekunden. Sie wählen Tabellengröße n = 3600 und die Hashfunktion h(t) = t mod 3600.

1. Warum ist das katastrophal?
2. Was würde passieren, wenn Sie n = 3607 (Primzahl) wählen und was müssten Sie beachten?

Übung

2.4. Hashwerte berechnen I

Berechnen Sie:

1. $h_{257}^{mod}(1\,000)$
2. $h_{257}^{mul}(1\,000)$

2.5. Hashwerte berechnen II

Berechnen Sie:

1. $h_{263}^{mod}(10\,000)$
2. $h_{263}^{mul}(10\,000)$

3. Hashing in Programmiersprachen

Hashing in Python

Hashing in Java

4. Hashing in Python

Verwendung von Hashes in Python

- Bei der Speicherung von Objekten in Sets und Dictionaries verwendet Python Hashes.
- Sobald ein Objekt in einem Set oder Dictionary gespeichert wird, darf der Objektzustand (zumindest im Hinblick auf die Hashfunktion) nicht mehr verändert werden!
- Der Hashwert eines (nicht veränderlichen) Objekts kann mit der Funktion `hash()` berechnet werden.
- Eigene Objekte in Sets und Dictionaries speichern:
  - Um benutzerdefinierte Objekte in einer Hashmap zu speichern, müssen wir die Methoden __hash__ und __eq__ implementieren.
  - Zu beachten:
    - Hashwerte *müssen für gleiche Objekte gleich sein*.
    - Hashwerte *für unterschiedliche Objekte sollten unterschiedlich sein*.

Beispielklasse `Person`

```
1  class Person:
2      def __init__(self, name, age):
3          self.name = name
4          self.age = age
5
6      def __eq__(self, other):
7          if isinstance(other, Person):
8              return  self.name == other.name and \
9                      self.age == other.age
10         return False
11
12     def __hash__(self):
13         return hash((self.name, self.age))
```

Verwendung der Klasse `Person`

```
1  person1 = Person("Alice", 30)
2  person2 = Person("Bob", 25)
3  person3 = Person("Alice", 30) # gleiche Werte wie "person1"
```

Beispielausgabe

```
>>> person1
<__main__.Person object at 0x101474c20>
>>> person2
<__main__.Person object at 0x1013daad0>
>>> person3
<__main__.Person object at 0x1013db110>
```

Speicherung von `Person`-Objekten in einem Set

```
1  people = {person1, person2, person3}
```

Ausgabe des Sets

```
1  for p in people: print(p.name)
```

Ausgabe

```
Bob
Alice
```

Verwendung der hash-Funktion

```
print(hash(person1))
print(hash(person2))
print(hash(person3))
```

Beispielausgabe

```
3529483511948588452
-9048922068811934735
3529483511948588452
```

In Python ist die Ausgabe der Funktion `hash()` nach jedem Neustart der Pythonumgebung unterschiedlich, da die Hashfunktion einen Zufallswert enthält, der bei jedem Neustart neu generiert wird.

Beispielklasse `Person` mit konstantem Hashwert

```
class PersonWithBadHash:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __eq__(self, other):
        if isinstance(other, Person):
            return  self.name == other.name and \
                    self.age == other.age
        return False

    def __hash__(self):
        return 1 # immer der gleiche Hashwert
```

Die Verwendung des Alters der Person als Hashwert wäre in den allermeisten Fällen auch keine gute Idee, da es (vermutlich) viele Hashkollisionen geben würde.

Verwendung einer Klasse mit einer konstanten Hashfunktion

```
person1 = Person("Alice", 30)
person2 = Person("Bob", 25)
person3 = Person("Alice", 30)
people = {person1, person2, person3}
print(hash(person1))
print(hash(person2))
print(hash(person3))
print(" ".join(map(lambda p: p.name, people)))
```

Beispielausgabe

```
1
1
1
Alice Bob
```

⚠ Warnung

Die Verwendung einer konstanten Hashfunktion ist in der Regel keine gute Idee, da sie die Effizienz von Hashmaps ganz erheblich beeinträchtigen kann.

✎ Übung

4.1. Eine Klasse zur Repräsentation von Studierenden.

Die Klasse `Student` soll:

- die Attribute/Properties `name` und `matriculation_number` haben.
- die Methoden `__eq__` und `__hash__` sinnvoll/korrekt definieren

Aufgaben:

1. Erzeugen Sie drei `Student`-Objekte und speichern Sie diese in einem Set.
2. *Fragen Sie sich wie sie effizient den Hashwert berechnen können.*
3. Geben Sie die Namen der Studierenden aus.
4. Was passiert, wenn Sie — *nachdem Sie ein Student Objekt dem Dictionary hinzugefügt haben* — den Namen des Studenten ändern?

   Schreiben Sie entsprechenden Code, um Ihre Annahme zu überprüfen!

Rumpfimplementierung

```
class Student:
    def __init__(self, ... ):
        raise NotImplementedError("TODO")

    def __eq__(self, other):
        raise NotImplementedError("TODO")

    def __hash__(self):
        raise NotImplementedError("TODO")
```

5. Hashing in Java

Verwendung von Hashes in Java

- Bei der Speicherung von Objekten in `Set`s und `HashMap`s/`Hashtable`s verwendet Java die Hashwerte, die von der Methode *`hashCode()`* zurückgegeben werden.
- Sobald ein Objekt in einem `Set` oder einer `Map` gespeichert wird, darf der Objektzustand (zumindest im Hinblick auf die Hashfunktion) nicht mehr verändert werden!
- Eigene Objekte in `Set`s und `Map`s speichern:
  - Um benutzerdefinierte Objekte in einer `HashMap` zu speichern, müssen wir die Methoden `boolean` *`equals`*`(Object o)` und `int hashCode()` implementieren.
  - Zu beachten:
    - Hashwerte *müssen für gleiche Objekte gleich sein*.
    - Hashwerte *für unterschiedliche Objekte sollten unterschiedlich sein*.

Beispielklasse `Person`

```java
class Person {
    private String name;
    private int age;
    Person(String name, int age) { this.name = name; this.age = age; }

    public boolean equals(Object o) {
        if (o instanceof Person) { // Alt. compare class objects
            Person other = (Person) o;
            return this.name.equals(other.name) && this.age == other.age;
        }
        return false;
    }

    public int hashCode() { return java.util.Objects.hash(name, age); }
}
```

※ Hinweis

In einem realen Project würden wir ein `record` statt einer Klasse verwenden; die Implementierung wäre praktisch identisch und würde automatisch generiert werden.

```java
record Person(String name, int age) {}
```

Verwendung der Klasse `Person`

```java
var person1 = new Person("Alice", 30);
var person2 = new Person("Bob", 25);
var person3 = new Person("Alice", 30); // gleiche Werte wie "person1"
```

Beispielausgabe

```
person1 ⟹ Person@750e297f // the addresses are memory addresses
person2 ⟹ Person@1fb0e5
person3 ⟹ Person@650e893b
```

Speicherung von `Person`-Objekten in einem Set

```java
var set = new HashSet<Person>();
set.add(person1);
set.add(person2);
set.add(person3);
```

```
// throws IllegalArgumentException:
var people = Set.of(person1, person2, person3)
```

Ausgabe des Sets

```
for (var p : people) System.out.println(p.name);
```

Ausgabe

```
Bob
Alice
```

Verwendung der hashCode-Funktion

```
System.out.println(person1.hashCode());
System.out.println(person2.hashCode());
System.out.println(person3.hashCode());
```

Beispielausgabe

```
1963862399
2076901
1963862399
```

Beispielklasse Person mit konstantem Hashwert

```
class PersonWithBadHash {
    String name;
    int age;
    PersonWithBadHash(String name, int age) { this.name = name; this.age = age; }

    public boolean equals(Object o) {
        if (o instanceof PersonWithBadHash) {
            PersonWithBadHash other = (PersonWithBadHash) o;
            return this.name.equals(other.name) && this.age == other.age;
        }
        return false;
    }

    public int hashCode() { return 1; /* immer der gleiche Hashwert */ }
}
```

Die Verwendung „nur“ des Alters der Person als Hashwert wäre in den allermeisten Fällen auch keine gute Idee, da es (vermutlich) viele Hashkollisionen geben würde.

Verwendung einer Klasse mit einer konstanten Hashfunktion

```
var person1 = new PersonWithBadHash("Alice", 30);
var person2 = new PersonWithBadHash("Bob", 25);
System.out.println(person1.hashCode());
System.out.println(person2.hashCode());
var people = Set.of(person1, person2);
people.forEach(p -> System.out.println(p.name));
```

Beispielausgabe

```
1
1
1
Alice Bob
```

⚠ Warnung

Die Verwendung einer konstanten Hashfunktion ist in der Regel keine gute Idee, da sie die Effizienz von Hashmaps ganz erheblich beeinträchtigen kann.

✎ Übung

5.1. Eine Klasse zur Repräsentation von Studierenden.

Die Klasse `Student` (nutzen Sie *hier* kein `record`) soll:

- Die Attribute/Properties `name` und `matriculationNumber` haben.
- Die Methoden *`equals`*`(Object)` und *`hashCode`*`()` sinnvoll/korrekt definieren.

Aufgaben:

1. Erzeugen Sie drei `Student`-Objekte und speichern Sie diese in einem Set.
2. *Fragen Sie sich wie sie effizient den Hashwert berechnen können.*
3. Geben Sie die Namen der Studierenden aus.
4. Was passiert, wenn Sie — *nachdem Sie ein* `Student` *Objekt einem* `HashSet` *hinzugefügt haben* — den Namen des Studenten ändern?

   Schreiben Sie entsprechenden Code, um Ihre Annahme zu überprüfen!

Rumpfimplementierung

```java
import java.util.HashSet;

class Student {
    private String name;
    private int matriculationNumber;

    public Student(String name, int matriculationNumber) {
        this.name = name;
        this.matriculationNumber = matriculationNumber;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getMatriculationNumber(){ return matriculationNumber; }
    public void setMatriculationNumber(int matriculationNumber) {
        this.matriculationNumber = matriculationNumber;
    }

    @Override public boolean equals(Object o) {
        throw new UnsupportedOperationException("TODO");
    }

    @Override public int hashCode() {
        throw new UnsupportedOperationException("TODO");
    }
}
```

6. Hashtabellen (🇺🇸 *Hashmaps* oder 🇺🇸 *Dictionaries*)

Grundlagen von Hashtabellen

Das Grundprinzip von Hashtabellen ist einfach:

- Im Vorfeld wird ein Array `A` einer Größe n angelegt,

  Ziel: Die Größe des Arrays übersteigt die erwartete Belegung deutlich.
- Daten mit einem Schlüssel k werden dann an der Position `A[`h`(k)]` gespeichert - oder an einer Ersatzposition.
- Sollte die Belegung zu groß werden, wird das Array vergrößert und die Elemente werden (ggf.) neu bzw. wieder gehasht.
- Sollten zwei Schlüssel den gleichen Hash haben (d. h. $h(x_1) = h(x_2)$), dann wird eine Kollisionsauflösung benötigt.

Belegung von Hashtabellen

Die Belegung von Hashtabellen ist für die Effizienz entscheidend.

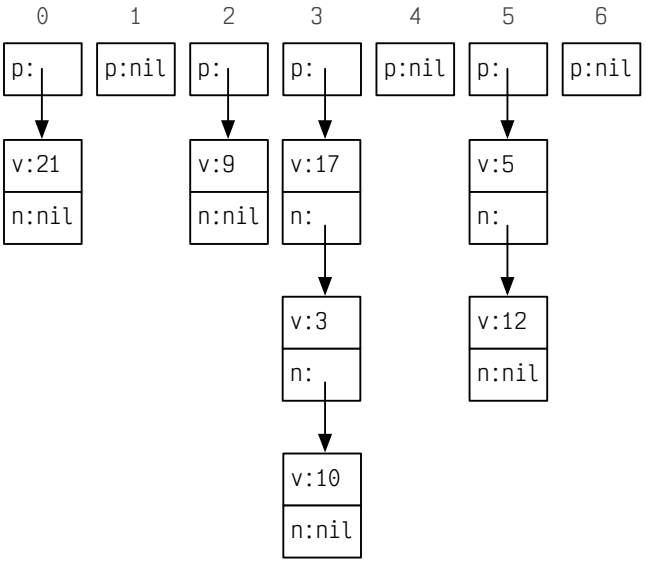
Definition: Hashtabelle

Ein Array A der Kapazität n mit einer Hashfunktion h_n wird $Hashtabelle(A, h_n)$ genannt.

Sind zu einem Zeitpunkt m (erste) Felder belegt, so hat die $Hashtabelle(A, h_n)$ eine *Belegung* von $\alpha = \frac{m}{n}$.

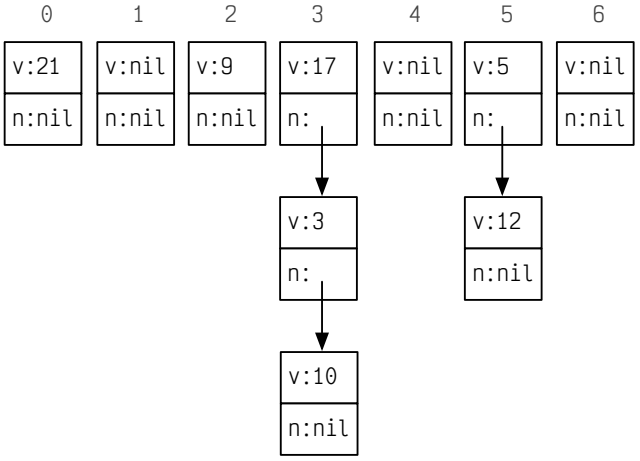
Verkettete Hashtabellen

Separate Verkettung



Die *separate Verkettung* verwendet eine $Hashtabelle(A, h_n)$, mit einem Array A, das aus Zeigern auf einfach verkettete Listen besteht, dessen Schlüssel der Einträge alle den gleichen Hashwert besitzen, oder die `nil` sind, wenn kein Eintrag bisher mit dem jeweiligen Hashwert vorhanden ist.

Direkte Verkettung



Die *direkte Verkettung* verwendet eine $Hashtabelle(A, h_n)$, bei der das Array A aus Knoten einer einfach verketteten Liste besteht, dessen Wert nil ist, wenn unter dem Hashwert noch nichts gespeichert wurde.

Ein Eintrag mit Schlüssel k wird der verketteten Liste zugeordnet, die in $A[h_n(k)]$ verlinkt ist oder startet, und kann entsprechend hinzugefügt, gelöscht und gefunden werden.

Offene Adressierung

Definition

Soll der $Hashtabelle(A, h_n)$ mit einem Array A ein Datensatz mit Schlüssel k hinzugefügt werden soll, so erfolgt dies in $A[h_n(k)]$ wenn dieser Eintrag noch nicht belegt ist.

Ansonsten werden $i = 1, ..., n-1$ weitere Positionen $A[g_n(k, i)]$ geprüft.

Strategien

Lineares-Sondieren:

Das Array wird linear durchsucht.

$$g_n^{lin}(k, i) = (h_n(k) + i) \ mod \ n$$

Quadratisches-Sondieren:

Das Array wird quadratisch steigend durchsucht.

$$g_n^{quad}(k, i) = (h_n(k) + i^2) \ mod \ n$$

Doppeltes-Hashing:

Das Array wird mit Hilfe einer zweiten Hashfunktion:

$$h_n^{'}(k) = (k \ mod \ (n-2)) + 1$$

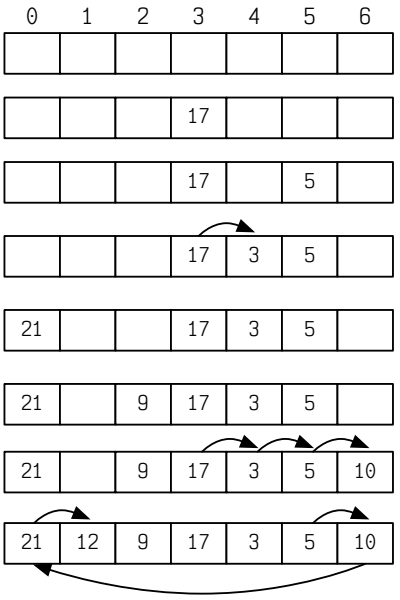
durchsucht.

$$g_n^{doppel}(k, i) = (h_n(k) + i \cdot h_n^{'}(k)) \ mod \ n$$

Beispiel Offene Adressierung (Hashing: $h(x)_n = x \bmod 7$)

Lineare Sondierung

Hinzufügen von (17, 5, 3, 21, 9, 10, 12)

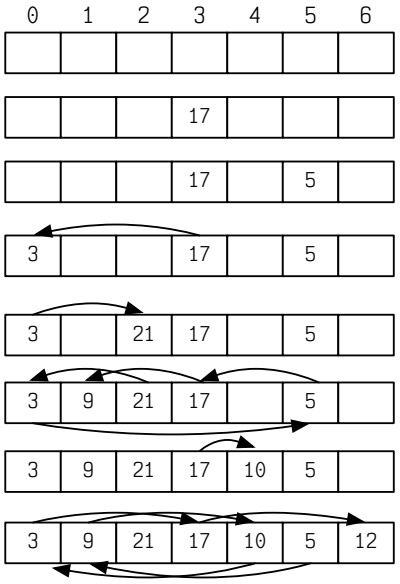


Beispiel: Hinzufügen von 3

1. Berechnung von $h_n(3) = 3 \bmod 7 = 3$ (belegt)
2. Berechnung von $g_n(3, 1) =$
   $(h_n(3) + 1) \bmod 7 = 4 \bmod 7 = 4$ (frei)

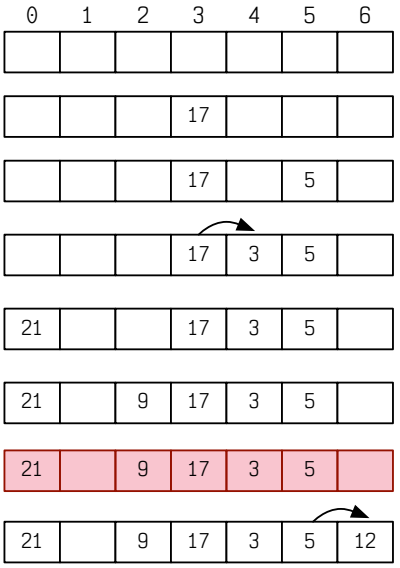
Doppeltes-Hashing

Hinzufügen von (17, 5, 3, 21, 9, 10, 12)



Quadratische Sondierung

Hinzufügen von (17, 5, 3, 21, 9, 10, 12)



Achtung!

Für den Wert 10 wird kein Platz gefunden!

$(10 \ mod \ 7 = 3)$

1. $3 + 0^2 \ mod \ 7 = 3$
2. $3 + 1^2 \ mod \ 7 = 4$
3. $3 + 2^2 \ mod \ 7 = 0$
4. $3 + 3^2 \ mod \ 7 = 5$
5. $3 + 4^2 \ mod \ 7 = 5$
6. $3 + 5^2 \ mod \ 7 = 0$

7. $3 + 6^2 \ mod \ 7 = 4$

✎ Übung

6.1. Werte in kleine Hashtabelle einfügen

Belegen Sie eine Hashtabelle mit $n = 5$ Feldern mit den Werten 37, 18, 32 und 24 auf Basis von $h_5^{mod}(x)$ mit linearer Sondierung, quadratischer Sondierung und doppeltem Hashing mit $h_5^{'}(x) = (x \ mod \ 3) + 1$.

6.2. Werte in größere Hashtabelle einfügen

Belegen Sie eine Hashtabelle mit $n = 11$ Feldern mit den Werten 37, 49, 26 und 39 auf Basis von $h_{11}^{mod}(x)$ mit linearer Sondierung, quadratischer Sondierung und doppeltem Hashing mit $h_{11}^{'}(x) = (x \ mod \ 9) + 1$.

Angriffe auf algorithmische Komplexität

Julian Wälde and Alexander Klink reported that the `String.hashCode()` hash function is not sufficiently collision resistant.

`hashCode()` value is used in the implementations of [Java 6] HashMap and Hashtable classes. A specially-crafted set of keys could trigger hash function collisions, which can degrade performance of HashMap or Hashtable by changing hash table operations complexity from an expected/average O(1) to the worst case O(n). Reporters were able to find colliding strings efficiently using equivalent substrings and meet in the middle techniques. This problem can be used to start a denial of service attack against applications that use untrusted inputs as HashMap or Hashtable keys. An example is a web application server that may fill hash tables with data from HTTP request. A remote attack could use that to make JVM use excessive amount of CPU time by sending a POST request with large amount of parameters which hash to the same value.

—[Abbreviated Version] Jan Lieskovsky 2011-11-01